

Towards Geometry-Consistent Federated Graph Learning

Yuecen Wei
School of Software
Beihang University
Beijing, China
weiyu@buaa.edu.cn

Zhiyu Zhuang
School of Computer Science and
Engineering
Beihang University
Beijing, China
zhuangzy25@buaa.edu.cn

Yisen Gao
Computer Science and Engineering
The Hongkong University of Science
and Technology
Hongkong, China
ygaodi@cse.ust.hk

Xingcheng Fu
Key Lab of Education Blockchain and
Intelligent Technology
Guangxi Normal University
Guilin, China
fuxc@gxnu.edu.cn

Qingyun Sun
School of Computer Science and
Engineering
Beihang University
Beijing, China
sunqy@buaa.edu.cn

Ziwei Zhang
School of Computer Science and
Engineering
Beihang University
Beijing, China
zwzhang@buaa.edu.cn

Tianyu Wo
School of Software
Beihang University
Beijing, China
woty@act.buaa.edu.cn

Chunming Hu*
School of Software
Beihang University
Beijing, China
hucm@buaa.edu.cn

Abstract

Federated Graph Learning (FGL) enables the collaborative training of graph neural networks without data leaving its local domain, thereby supporting risk-sensitive applications such as cross-platform personalized recommendations and financial risk control. However, its potential in heterogeneous cross-client environments has not been fully explored. The existing FGL methods are limited by the representation ability of Euclidean space for complex topological graph data, and are confronted with geometric heterogeneity caused by topological heterogeneity, making it difficult to capture the inherent curvature and hierarchical structure of graph data in the manifold space. When there are significant differences in community structure, homophily/heterophily, and local curvature among clients, biases can be amplified, knowledge alignment is hindered, and this further induces global aggregation distortion and insufficient personalization adaptation. In this paper, we propose a **Geometry-consistent Federated** graph learning framework (GeoFed), which achieves geometric alignment and structural adaptation while preserving privacy constraints, thereby mitigating aggregation bias. Specifically, we design a Geometry-Infused Local Learner that constructs edge weights and hierarchical representations for local training based on hyperbolic models, thereby enhancing the geometric constraints and importance of information flow, and achieving key steps of geometric infusion. Subsequently, we introduce a Geometry-Consistent Aggregator that collects client

model parameters and hyperbolic embeddings, and constructs a dual-kernel affinity matrix as a soft transfer among geometrically similar clients to route client-specific knowledge. Extensive experiments demonstrate that GeoFed achieves stable improvements over optimal baselines across multiple datasets, validating the effectiveness and generality of geometric consistency in FGL.

CCS Concepts

• **Computing methodologies** → **Neural networks; Distributed algorithms**; • **Mathematics of computing** → **Graph algorithms**.

Keywords

Federated Graph Learning; Geometric Representation Learning; Hyperbolic Geometry Model

ACM Reference Format:

Yuecen Wei, Zhiyu Zhuang, Yisen Gao, Xingcheng Fu, Qingyun Sun, Ziwei Zhang, Tianyu Wo, and Chunming Hu. 2026. Towards Geometry-Consistent Federated Graph Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792655>

1 Introduction

Graph-structured data has become ubiquitous across diverse domains [59, 65], from social networks that model user relationships and financial systems that map transactional connections to modeling for the World Wide Web (e.g., representing hyperlink structures between web pages) [28, 76], enabling powerful data-driven insights for applications like personalized recommendation [8], financial risk control [67], and drug discovery [26]. The rich relational information embedded in graph data often requires collaboration across organizations or platforms to unlock its full value, as isolated local datasets lack the scale and diversity needed for robust model training [13]. However, direct data sharing across clients is

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792655>

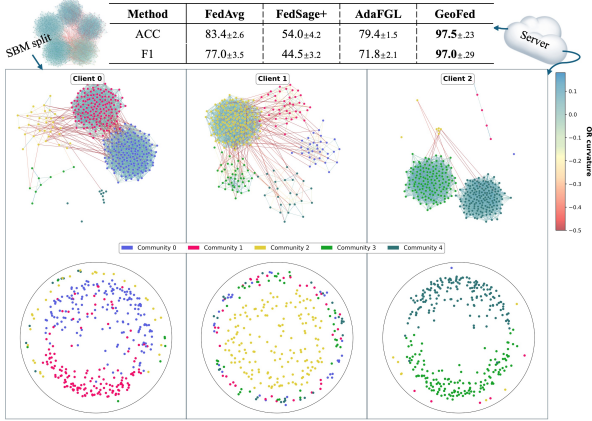


Figure 1: A toy example distribution on geometric space for the graph generated from the Stochastic Block Model. The first row illustrates the community structure, where the edge colors indicate Ricci weights. The second row shows the distribution of nodes on the Poincaré disk.

severely constrained by strict privacy regulations (e.g., GDPR [47]) and growing concerns over data leakage, creating a critical barrier to collaborative graph learning [38, 39, 64].

Federated Graph Learning (FGL) has emerged to enable training graph neural networks (GNNs) [20, 58] across multiple clients without exposing raw data [36]. While FGL faces challenges like privacy-preserving aggregation [42] and personalization in non-IID settings [19, 66], a critical issue is topological heterogeneity across client graphs [49]. Some works address this heterogeneity from a statistical perspective by employing weighted averaging [42, 80], proximal regularization [9, 27], and cluster-based personalization [14, 48], or by incorporating structural priors or spectral and label propagation statistics into aggregation to improve alignment [10]. However, these methods are limited to Euclidean space and are insufficient for adequately representing topological heterogeneity.

On the other hand, hyperbolic geometry, characterized by its exponential growth property resulting from negative curvature, provides a natural low-distortion space for graph embeddings. In recent years, hyperbolic representation learning has achieved significant improvements in tasks such as node classification [6, 25], graph generation [11, 63], and community detection [44]. Motivated by these advantages, we seek to interpret and address topological heterogeneity in FGL from the perspective of low-distortion hyperbolic geometry.

As illustrated in Fig. 1, we visualize the graph data generated from the Stochastic Block Model (SBM). To characterize its geometric properties, we employ Ricci curvature to capture local geometric features (edge colors where warm signify high dispersion between nodes, meaning the distances between nodes are greater than expected in Euclidean space, while cool colors indicate the opposite), and use the Poincaré ball model to represent the global geometric structure (where nodes closer to the edge exhibit stronger specificity, contributing more significantly to the model’s personalization). Our analysis reveals that, beyond the commonly discussed

heterogeneity in node count and topological distribution, there exists **geometric heterogeneity** caused by topological differences, which describes the differences in the underlying geometric properties of cross-client graphs in a non-Euclidean manifold space, and is the intrinsic mapping of topological heterogeneity on the manifold. For example, from a global perspective, Client 2 exhibits a dominant yellow-node cluster, whereas Client 3 presents two clusters of comparable size, resulting in clear discrepancies in their global geometric characteristics. Although previous studies have sought to capture richer geometric information using various geometric metrics and have introduced regularization constraints to align global representations in manifold space, thereby mitigating inconsistent biases [1, 40, 61], there remains a lack of effective strategies to accurately perceive topological geometric signals from client-side local graph data and translate them into personalized aggregation at the federated level. This limitation hinders the mitigation of geometric inconsistency across clients and the training bias caused by topological differences through geometric heterogeneity.

Underlying geometric factors determine the way to achieve geometry-consistent FGL. However, it faces two challenges: (1) Personalized Geometric Representation: How can we effectively model and capture the unique geometric properties of each client? This involves accurately reflecting the local structural characteristics specific to each client’s graph data. (2) Cross-client Global Geometric Alignment: How can we align and integrate the features with heterogeneous geometric characteristics contributed by each client at the federated aggregation level? This requires reconciling differences across clients to ensure a coherent global representation.

Present work. To address the aforementioned challenges, we propose GeoFed¹, a geometry-consistent federated graph learning framework that achieves geometric alignment and structural adaptation under privacy constraints. GeoFed consists of two key components: the Geometry-Infused Local Learner (GILL) and the Geometry-Consistent Aggregator (GCA). GILL leverages hyperbolic models to learn edge-level curvature weights and node importance, enhancing the distinguishability of channels and hierarchies, and emphasizing critical topology-sensitive relationships. GCA estimates dual-kernel weights from hyperbolic embeddings and constructs a personalization matrix based on geometric similarity to robust aggregation, balancing commonality and specificity. Extensive experiments on real-world benchmarks demonstrate that GeoFed outperforms baselines, achieving at least a 14% performance improvement in client distributions with strong community characteristics. We summarize our contributions as follows:

- We propose a novel FGL framework, GeoFed, aimed at geometric consistency by integrating dual-path geometric signals across client injection and server fusion, forming a generalizable federated geometric alignment strategy.
- GeoFed design geometric enhancements for graph structures, driven by geometric similarity to facilitate personalized and robust federated-level weight fusion.
- Comprehensive experiments on multiple real-world datasets validate the effectiveness and superiority of GeoFed, and the necessity of geometric knowledge and fusion strategies.

¹Code available: <https://github.com/RingBDStack/GeoFed>

2 Related Work

2.1 Federated Graph Learning

For clients with privatized graph structure data, FGL aims to federate the optimization of structural information and jointly train a central server model to leverage structural modeling for potential data associations across clients effectively [13, 36, 38]. Classical federated optimization methods [27, 42] are susceptible to aggregation mismatches and performance fluctuations in graph tasks caused by data distribution, topology and label distribution shifts. To mitigate these issues, existing methods can be categorized into global model training and personalization. On one hand, current techniques utilize client graph statistics (such as degree, community, and homophily) [41, 54, 66] or graph distillation signals for weighting [51, 83], constructing global pseudo-labels [2, 30], or performing cross-client consistency regularization [79, 81] to achieve topology-aware aggregation and global structural gains. On the other hand, each client trains a tailored local personalized model to enhance client performance through local adaptation, parameter decoupling, or multi-task learning [2, 29, 37, 54]. Notably, client clustering is a common personalization strategy, where clients with similar data distributions are grouped, and clients within the group share the same model parameters [5, 34, 66].

However, they are limited by embedding in Euclidean space, lacking sufficient federated-level modeling of geometric factors such as curvature and hierarchy, so struggling to address cross-client curvature statistics and hyperbolic position alignment in terms of geometric consistency.

2.2 Hyperbolic Geometry Deep Learning

Hyperbolic geometry [21, 32, 71] extends Euclidean deep operators to spaces that efficiently accommodate hierarchical, heterophily, and long-tailed structures by performing representations and computations on negatively curved manifolds, such as the Poincaré ball and Lorentz model [22, 45, 46, 55, 62]. Benefiting from the powerful feature learning and aggregation capabilities of GNNs, Hyperbolic Graph Neural Networks (HGNNs) [35] aim to integrate the message-passing paradigm with non-Euclidean manifolds [6, 72, 78] to leverage hyperbolic space's ability to perceive the geometric structure of data. In centralized scenarios, HGNNs are based on Möbius operations [57], projecting hyperbolic embeddings to local Euclidean tangent spaces where linear transformations, residuals, and attention operations are performed in this flat space, and then projecting the results back to the hyperbolic manifold, offering advantages over GNN methods on hierarchical or tree-like data [12, 16, 68, 70, 75]. Advanced work introduces new structural knowledge driven by curvature to enhance inter-community propagation, achieve multi-scale structural adaptation in mixed curvature spaces, and explore scalability for large graphs [15, 52]. In distributed scenarios, some methods attempt to utilize different geometric metrics, such as cosine similarity [69], Power Diagram [24], and Hessian matrices [82], to obtain potential structural connections and follow the federated learning paradigm for regularized representation and normalized aggregation [1, 40, 53].

However, existing methods confine geometric constraints to the client side, lacking fusion of multi-source geometric signals and personalized transmission.

3 Preliminary

Notation. A graph is denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the node set and $\mathcal{E}$ is the edge set. Let $e_{ij} = (v_i, v_j)$ denotes an edge between node $v_i \in \mathcal{V}$ and $v_j \in \mathcal{V}$. A series of nodes and edges is represented as an adjacency matrix $\mathbf{A} \in \{0, 1\}^{N \times N}$ and node features $\mathbf{X} \in \mathbb{R}^{N \times d}$, where $N = |\mathcal{V}|$ is the number of nodes and d is the dimensionality.

DEFINITION 1 (FEDERATED GRAPH LEARNING). *Given $\mathcal{K}$ clients $\mathcal{S} = \{s_k\}_{k=1}^{\mathcal{K}}$, where each client k privately owns a graph dataset $\mathcal{G}_k = \{(\mathbf{X}_i, \mathbf{Y}_i)_{i=1}^{N_k}\}$ and $\mathbf{Y}_i$ is the corresponding label. For a semi-supervised node classification task, the goal of FGL is to collaboratively train GNN f_θ with parameters θ without exchanging raw data. The optimization objective is:*

$$\min_{\theta} \frac{1}{N} \sum_{k=1}^{\mathcal{K}} \sum_{i=1}^{N_k} \mathcal{L}(f_\theta(\mathbf{X}_i), \mathbf{Y}_i), \quad (1)$$

where $\mathcal{L}$ is the loss function.

FedAvg [42] is a fundamental method in Federated Learning that enables decentralized model training across different devices while preserving data privacy. It is initiated by a central server distributing global model parameters θ^t . Clients independently perform local model updates θ_k^t using stochastic gradient descent. These updates are collected by the server and aggregated through averaging to form a new global model θ^{t+1} . This process iterates for t rounds until convergence, refining the global model $\theta^{t+1} = \frac{1}{\mathcal{K}} \sum_{k=1}^{\mathcal{K}} \theta_k^t$. Following the above paradigm, we will introduce geometric knowledge to achieve more advanced consistency aggregation, ensuring good generalization across all client distributions while allowing personalization to adapt to each client's geometric and statistical differences.

Hyperbolic Geometry Model. Hyperbolic geometry is a Riemannian manifold characterized by constant negative curvature, which is crucial for modeling graph data exhibiting tree-like structures or power-law distributions. The primary operations include geodesics and exponential/logarithmic mappings [6, 71].

The *Geodesic* provides the shortest length of curve γ , its distance between points $\mathbf{u}, \mathbf{v} \in \mathcal{M}$ is formally defined as $g_{\mathcal{M}}(\mathbf{u}, \mathbf{v})$. For a point $\mathbf{x} \in \mathcal{M}$ and its corresponding tangent vector $\mathbf{v} \in T_{\mathbf{x}}\mathcal{M}$, where $T_{\mathbf{x}}\mathcal{M}$ denotes the tangent space of manifold $\mathcal{M}$ at $\mathbf{x}$. There exists a unique geodesic $\gamma : [0, 1] \rightarrow \mathcal{M}$ such that $\gamma(0) = \mathbf{x}$ and $\gamma'(0) = \mathbf{v}$. The *exponential map* $\exp_{\mathbf{x}} : T_{\mathbf{x}}\mathcal{M} \rightarrow \mathcal{M}$ is then defined as $\exp_{\mathbf{x}}(\mathbf{v}) = \gamma(1)$, effectively “exponentiating” the tangent space direction into the manifold’s curved geometry. The *logarithmic map* $\log_{\mathbf{x}}$, defined as the inverse of $\exp_{\mathbf{x}}$, projects a point back onto its corresponding tangent vector in $T_{\mathbf{x}}\mathcal{M}$, such that this vector maps back to the original point under the exponential map.

We chose the *Poincaré ball model* as a modeling tool from among the five commonly used isometric models in hyperbolic geometry, because it can personalize the measurement of node hierarchical importance within clients based on the boundary stretching effect, enabling intuitive visualization and interpretability. Other models (such as the Lorentz model) can also be used as alternatives depending on the scenario requirements, but attention should be paid to the optimization issues of each model.

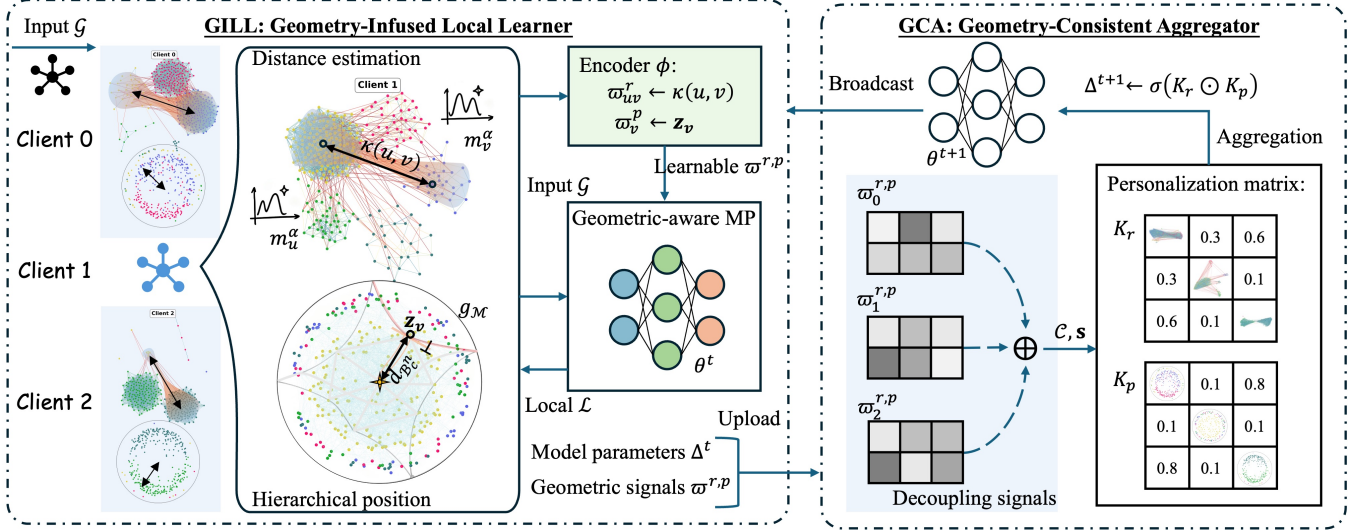


Figure 2: An overview of GeoFed. (1) GILL learns distance weights and hierarchical positions for local graph data based on a hyperbolic model, enhancing geometric constraints and information flow during message passing (MP). (2) GCA constructs a dual-kernel affinity matrix as a soft transfer between geometrically similar clients to route client-specific knowledge.

DEFINITION 2 (POINCARÉ BALL MODEL). Given a curvature parameter $c > 0$, Poincaré Ball $\mathcal{B}^n$ is a Riemannian manifold $(\mathcal{B}_c^n, g_x^{\mathcal{B}})$, where $\mathcal{B}_c^n$ is an n -dimensional ball of radius $1/\sqrt{c}$ and $g_x^{\mathcal{B}}$ is metric tensor. The Poincaré distance between the node pair (x, y) is defined as

$$d_{\mathcal{B}_c^n}(x, y) = \frac{2}{\sqrt{c}} \tanh^{-1}(\sqrt{c} \|\mathbf{x} \oplus_c \mathbf{y}\|), \quad (2)$$

where $\oplus_c$ is Möbius addition and $\|\cdot\|$ is the L_2 norm.

Ollivier Ricci Curvature. The Ricci curvature of a Riemannian manifold can measure the deviation of a curved manifold (e.g. hyperbolic space) from a flat manifold (Euclidean space) [23, 43]. We select the Ollivier Ricci (OR) curvature of random geometric graphs, which has been shown to converge to the Ricci curvature of the underlying Riemannian manifold [3], that is suitable for deep mining of local topological and geometric correlations, which aligns with the core scenario of this paper, where each client has a small-to-medium-scale network. Therefore, we use OR curvature to estimate the actual distance between nodes in manifold space, reflecting the true individuality of each node. Formally, for $\alpha \in [0, 1]$ and a node u with degree d_u , its probability distribution of neighbors is defined as:

$$m_u^\alpha(v) = \begin{cases} \alpha & \text{if } v = u \\ \frac{1-\alpha}{d_u} & \text{if } v \text{ is a neighbor of } u, \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

where α indicates the probability mass retained at node u itself, and the remainder is uniformly distributed over its neighborhood, which is typically set to 0.5.

DEFINITION 3 (OLLIVIER RICCI CURVATURE). Given the Wasserstein distance $\mathcal{W}(m_u^\alpha, m_v^\alpha)$ and the geodesic distance $g(u, v)$, the Ollivier Ricci curvature κ is defined as:

$$\kappa(u, v) = 1 - \frac{\mathcal{W}(m_u^\alpha, m_v^\alpha)}{g(u, v)}, \quad (4)$$

Here, the Wasserstein distance can be computed via linear programming [33].

4 Method

In this section, we will elaborate on the proposed GeoFed and its framework as illustrated in Fig. 2. The server first broadcasts the global model to participating clients. Then, we perform geometric infusion at the client side by introducing complementary hyperbolic signals into the local model, which respectively affect the information pathway weights of edges and the importance weights of nodes. Subsequently, geometric fusion is conducted at the server side, and a personalization matrix is then constructed based on geometric similarity, collectively generating the global parameters to be distributed.

4.1 Geometry-Infused Local Learner

In FGL, the statistical distribution and topological geometry of client graphs are often heterogeneous, making it difficult for simple feature- or label-based local training to capture deep structural differences. To align the optimal filter with the optimal embedding geometry, we design two geometric signals at the client side. On the one hand, we calculate the graph's OR curvature following Eq. (4) as an estimate of the distance between nodes, emphasizing the bottleneck and clustering properties of pathways. Then, the curvature is mapped to learnable edge weights:

$$\tilde{\omega}_{uv}^r = \sigma(\mathbf{w}_r \kappa(u, v) + \mathbf{b}_r), \quad \omega_{uv}^r = \frac{\tilde{\omega}_{uv}^r}{\sum_{x \in \mathcal{N}(u)} \tilde{\omega}_{ux}^r}, \quad (5)$$

where $\mathcal{N}$ is the neighbor set of node u and σ is activation function.

On the other hand, considering that many real-world graphs exhibit hierarchical and long-tailed degree distribution characteristics, we leverage the ability of hyperbolic space to accommodate exponentially growing neighborhoods in low dimensions with high

fidelity. According to Definition 2, the nodes are mapped to the Poincaré ball to explicitly represent hierarchical positions and importance, providing interpretable geometric information for subsequent aggregation and personalization. This process obtains hyperbolic embeddings through tangent space affine transformations and exponential mapping:

$$\xi_v = \mathbf{w}x_v + \mathbf{b}, \mathbf{z}_v = \exp_0^c(\xi_v) = \tanh(\sqrt{c}\|\xi_v\|) \frac{\xi_v}{\sqrt{c}\|\xi_v\|}. \quad (6)$$

Furthermore, node importance is generated using the embedding radius and angle to maintain hierarchical semantic consistency:

$$\omega_v^p = \sigma(\phi_p(\mathbf{z}_v)) \in (0, 1)^d, \quad (7)$$

where ϕ_p represents the hyperbolic embedding encoder, and here we employ the Multilayer perceptron.

Geometric-aware message passing. Without changing the geometry of the operator, we introduce the above weights, and the embedding of the $l + 1$ -th layer is:

$$\mathbf{h}_v^{(l+1)} = \sigma((\omega_v^p \odot \mathbf{w}_v \mathbf{h}_v^{(l)}) + \sum_{\mu \in \mathcal{N}(v)} \omega_{\mu v}^r \cdot (\mathbf{w}_\mu \mathbf{h}_\mu^{(l)})), \quad (8)$$

where $\odot$ denotes the Hadamard product. Next, we conduct local training, and the local objective function is given by a semi-supervised node classification cross-entropy loss:

$$\mathcal{L} = \frac{1}{|\mathcal{V}_L|} \sum_{v \in \mathcal{V}_L} -\log \hat{y}_{v, y(v)} + \mathcal{R}_{\text{geo}}, \quad (9)$$

where $\mathcal{V}_L$ is the set of labeled nodes, y_v is the true label, $\hat{y}_v$ is the predicted probability for node v , and $\mathcal{R}_{\text{geo}}$ represents the geometric regularization term, which encourage structural diversity and ensure training stability.

Finally, to maintain privacy constraints, we do not directly upload the raw OR curvature and Poincaré embeddings, which represent topological distances and node hierarchies, respectively. Instead, in addition to the model parameters from the clients, we upload low-dimensional embeddings of the geometric signals (ω_v^p and ω_v^r), which do not directly reflect specific geometric characteristics. If a malicious attacker attempts to infer individual client conditions under this framework, they would need to obtain the specific distribution of nodes or edges to decouple the geometric embeddings, but such information will not be provided.

4.2 Geometry-Consistent Aggregator

To construct a client-specific personalization matrix for dynamically routing knowledge among geometrically similar neighbors, upon receiving the geometric signals, the server first abstracts each client into a geometric signature. Specifically, let C_i denote the geometric centroid of client i in the Poincaré ball, representing its overall hierarchical position (e.g., core vs. periphery). Similarly, let $\mathbf{s}_i$ denote the aggregated Ollivier-Ricci curvature statistics, used to capture the topological structural features of the client (e.g., congestion or flatness).

Geometric neighborhood personalization matrix. To balance the commonalities and specificities of clients, we construct a dual-kernel personalization matrix based on node geometry (centroid distance) and edge geometry (statistical similarity). This serves as a mechanism for soft transfer among geometrically similar clients,

forming a personalized parameter routing that balances cross-client generalization and local customization. Specifically, clients with similar hierarchical positions (i.e., smaller Euclidean distances between centers) are assigned higher similarity scores, while clients exhibiting similar topological congestion patterns are considered structurally consistent. The two kernel functions are defined as follows:

$$\begin{aligned} K_p[i, j] &= \exp\left(-\|C_i - C_j\|^2 / \tau_p\right), \\ K_r[i, j] &= \exp\left(\cos(\mathbf{s}_i, \mathbf{s}_j) / \tau_r\right), \end{aligned} \quad (10)$$

where τ_p and τ_r are temperature parameters controlling the sensitivity to hierarchical and structural differences, respectively. Specifically, τ_p ensures that clients preferentially aggregate domain knowledge from hierarchically compatible peers, while τ_r regulates the tolerance for topological divergence, filtering out noise from clients with inconsistent curvature patterns. Finally, we fuse the two types of geometric information via the Hadamard product to represent the weight of model parameters transferred from client j to client i and obtain the personalization matrix:

$$\mathcal{A}_{ij} = \text{Norm}\left(K_p[i, j] \odot K_r[i, j]\right), \quad (11)$$

where $\text{Norm}(\cdot)$ denotes the row-wise normalization operation. Consequently, the personalized model update for client i is computed as a weighted combination of parameters from other clients:

$$\theta_i^{t+1} \leftarrow (1 - \lambda)\theta_i^t + \lambda \sum_{j \in \mathcal{S}_t} \mathcal{A}_{ij} \cdot \theta_j^t, \quad (12)$$

where $\mathcal{S}_t$ denotes the set of clients participating in round t , and λ is a hyperparameter controlling the mixing ratio between local and external knowledge. This personalization process functions as a form of soft geometric clustering, allowing each client to possess a unique, continuous neighborhood in the model space to weigh the contributions of others. By assigning high weights only to geometrically consistent peers, GeoFed effectively filters out biases introduced by structural noise and promotes the sharing of knowledge.

4.3 Complexity Analysis

Let n , m , c , and d denote the numbers of nodes, edges, classes, and feature dimensions. On the client side, the local computation follows the backbone GNN with cost $O(md + nd^2)$. The GeoFed-specific overhead only generates lightweight geometry summaries using two tiny MLPs on precomputed curvature and keys: Ricci edge embedding $O(m)$ and Poincaré node embedding $O(2n)$, yielding total additional $O(m + 2n)$ time and $O(m + n)$ memory. On the server side, computing client centers via log-map and curvature statistics costs $O(\mathcal{K}\bar{n} + 2\mathcal{K})$ ($\bar{n}$ is the average client node count). Building the dual kernels and the personalization matrix requires $O(3\mathcal{K}^2)$. Aggregating a single global model is $O(\mathcal{K}|\theta|)$; forming $\mathcal{K}$ personalized models is $O(\mathcal{K}^2|\theta|)$. Different from coupled-loss optimization strategies whose additional objectives are entangled with local training, GeoFed's extra computation consists of training-independent sparse and dense operations and small matrix multiplications, leading to low overhead.

Table 1: Accuracy and F1 score of the node classification task. “OOM” stands for out-of-memory. (Ten clients and ten runs. Result: average score $\pm$ standard deviation; Bold: the best of baseline model; Underline: runner-up.)

Model	Cora		CiteSeer		PubMed		ogbn-arxiv		Avg. Rank
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	
FedAvg [42]	81.44 $\pm$ 0.28	78.55 $\pm$ 0.41	69.19 $\pm$ 0.27	64.76 $\pm$ 0.29	85.69 $\pm$ 0.08	83.21 $\pm$ 0.07	62.27 $\pm$ 0.13	55.58 $\pm$ 0.19	7.0
FedSage+ [77]	81.27 $\pm$ 0.46	77.97 $\pm$ 0.54	69.15 $\pm$ 0.53	65.20 $\pm$ 0.54	<u>86.57 $\pm$ 0.10</u>	84.72 $\pm$ 0.12	<u>64.69 $\pm$ 0.13</u>	<u>59.15 $\pm$ 0.18</u>	3.3
FedGTA [30]	81.43 $\pm$ 0.36	78.42 $\pm$ 0.42	69.02 $\pm$ 0.52	64.54 $\pm$ 0.54	85.38 $\pm$ 0.06	82.82 $\pm$ 0.09	64.17 $\pm$ 0.08	58.73 $\pm$ 0.11	6.0
Fed-PUB [2]	82.07 $\pm$ 0.39	<u>79.66 $\pm$ 0.46</u>	70.18 $\pm$ 0.38	<u>66.15 $\pm$ 0.39</u>	86.08 $\pm$ 0.29	84.62 $\pm$ 0.34	63.90 $\pm$ 0.22	57.08 $\pm$ 0.40	3.3
FGSSL [18]	81.00 $\pm$ 0.26	78.26 $\pm$ 0.27	64.33 $\pm$ 0.46	61.76 $\pm$ 0.49	85.21 $\pm$ 0.09	83.42 $\pm$ 0.12	OOM	OOM	8.3
FedGL [7]	77.52 $\pm$ 0.51	73.64 $\pm$ 0.48	60.00 $\pm$ 0.79	56.35 $\pm$ 0.97	80.87 $\pm$ 0.47	79.07 $\pm$ 0.67	34.92 $\pm$ 1.79	24.74 $\pm$ 2.10	9.3
AdaFGL [29]	<u>81.81 $\pm$ 0.19</u>	79.89 $\pm$ 0.20	68.09 $\pm$ 0.80	65.63 $\pm$ 0.79	86.56 $\pm$ 0.12	<u>85.44 $\pm$ 0.18</u>	59.73 $\pm$ 0.23	51.96 $\pm$ 0.34	4.5
FGGP [60]	73.39 $\pm$ 0.99	71.12 $\pm$ 1.06	54.32 $\pm$ 1.46	50.32 $\pm$ 1.71	80.63 $\pm$ 0.43	78.37 $\pm$ 0.73	OOM	OOM	10.3
FedTAD [83]	81.55 $\pm$ 0.35	78.79 $\pm$ 0.49	69.31 $\pm$ 0.46	64.56 $\pm$ 0.71	85.69 $\pm$ 0.08	83.14 $\pm$ 0.09	61.17 $\pm$ 0.13	53.88 $\pm$ 0.20	6.3
FedIIH [74]	80.03 $\pm$ 0.68	49.93 $\pm$ 1.0	<u>70.66 $\pm$ 0.22</u>	56.63 $\pm$ 0.29	86.86 $\pm$ 0.11	73.97 $\pm$ 0.08	50.72 $\pm$ 0.12	52.38 $\pm$ 0.07	6.5
GeoFed(ours)	81.48 $\pm$ 0.39	78.68 $\pm$ 0.39	71.74 $\pm$ 0.26	69.98 $\pm$ 0.29	86.90 $\pm$ 0.11	86.42 $\pm$ 0.09	72.06 $\pm$ 0.04	70.12 $\pm$ 0.07	1.5

Model	Photo		Computers		CS		Physics		Avg. Rank
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	
FedAvg [42]	91.83 $\pm$ 0.00	90.42 $\pm$ 0.27	87.00 $\pm$ 0.00	85.22 $\pm$ 0.98	88.89 $\pm$ 0.00	87.37 $\pm$ 0.14	93.84 $\pm$ 0.00	93.48 $\pm$ 0.05	5.0
FedSage+ [77]	92.44 $\pm$ 0.33	<u>90.85 $\pm$ 0.44</u>	84.73 $\pm$ 1.31	81.71 $\pm$ 1.74	89.45 $\pm$ 0.18	87.30 $\pm$ 0.25	94.35 $\pm$ 0.07	93.96 $\pm$ 0.25	4.0
FedGTA [30]	91.88 $\pm$ 0.20	90.58 $\pm$ 0.25	85.41 $\pm$ 0.37	83.09 $\pm$ 0.49	89.36 $\pm$ 0.10	87.79 $\pm$ 0.13	93.90 $\pm$ 0.06	93.53 $\pm$ 0.04	4.8
Fed-PUB [2]	91.03 $\pm$ 0.14	89.08 $\pm$ 0.19	<u>88.12 $\pm$ 0.20</u>	<u>86.53 $\pm$ 0.19</u>	89.94 $\pm$ 0.15	87.82 $\pm$ 0.18	93.98 $\pm$ 0.11	93.36 $\pm$ 0.13	4.5
FGSSL [18]	70.75 $\pm$ 2.66	62.34 $\pm$ 3.23	66.07 $\pm$ 2.48	56.55 $\pm$ 3.26	88.18 $\pm$ 0.14	88.18 $\pm$ 0.17	OOM	OOM	8.7
FedGL [7]	72.46 $\pm$ 4.41	65.50 $\pm$ 4.75	64.80 $\pm$ 3.70	55.95 $\pm$ 4.34	83.40 $\pm$ 0.56	81.80 $\pm$ 0.60	91.71 $\pm$ 0.28	91.04 $\pm$ 0.38	8.0
AdaFGL [29]	90.79 $\pm$ 0.26	88.74 $\pm$ 0.35	85.48 $\pm$ 0.51	82.33 $\pm$ 1.02	91.32 $\pm$ 0.15	<u>89.87 $\pm$ 0.17</u>	95.07 $\pm$ 0.08	<u>94.70 $\pm$ 0.12</u>	3.3
FGGP [60]	74.15 $\pm$ 3.00	66.83 $\pm$ 3.44	62.41 $\pm$ 2.64	53.95 $\pm$ 2.60	82.00 $\pm$ 2.87	80.26 $\pm$ 3.57	92.00 $\pm$ 0.29	91.46 $\pm$ 0.23	9.3
FedTAD [83]	91.41 $\pm$ 0.27	89.78 $\pm$ 0.43	86.01 $\pm$ 0.32	83.62 $\pm$ 0.80	88.71 $\pm$ 0.13	87.54 $\pm$ 0.15	93.88 $\pm$ 0.09	93.54 $\pm$ 0.12	3.8
FedIIH [74]	91.61 $\pm$ 0.15	55.86 $\pm$ 0.94	83.88 $\pm$ 0.92	37.77 $\pm$ 0.71	<u>91.06 $\pm$ 0.12</u>	57.77 $\pm$ 0.81	<u>95.08 $\pm$ 0.09</u>	68.75 $\pm$ 0.37	7.8
GeoFed(ours)	<u>92.35 $\pm$ 0.24</u>	91.28 $\pm$ 0.27	88.56 $\pm$ 0.12	87.65 $\pm$ 0.16	90.88 $\pm$ 0.10	90.06 $\pm$ 0.14	95.11 $\pm$ 0.04	94.81 $\pm$ 0.05	1.5

5 Experiments

In this section, we evaluate GeoFed on multiple benchmark datasets, focusing on the following research questions: **RQ1**: How does the performance of GeoFed compare to state-of-the-art FGL methods? **RQ2**: What are the individual contributions of the designed geometry-guided and personalization modules? **RQ3**: How robust is GeoFed in scenarios with strong geometric heterogeneity? **RQ4**: How does it perform in terms of hyperparameters, convergence, and time consumption?

5.1 Experimental Settings

Datasets. We evaluate GeoFed on eight graph datasets, including three citation networks (Cora, CiteSeer, PubMed) [73], two co-author graphs (Co-author CS, Co-author Physics) [50], two e-commerce networks (Amazon Computers, Amazon Photo), and one large-scale benchmark (ogbn-arxiv) [17]. These datasets cover a wide range of graph scales, feature dimensions, and homophily levels, thus providing a comprehensive testbed for evaluating FGL under various heterogeneity conditions.

Baselines. To rigorously evaluate our framework, we adopt a series of FGL methods covering different learning paradigms, including FedAvg [42], which serves as the standard aggregation baseline without personalization. To measure how explicitly leveraging structural information mitigates the negative impact of subgraph

heterogeneity, we consider three topology-aware approaches: FedTAD [83], FedGL [7], and FedGTA [30]. For personalized aggregation frameworks FedPUB [2] and AdaFGL [29], we aim to assess how client-specific aggregation strategies enhance adaptability under non-IID data distributions. Moreover, we introduce a recent work, FedIIH [74], which focuses on modeling both inter- and intra-client heterogeneity by hierarchical modeling. Finally, self-supervised and generative approaches: FedSage+ [77], FGSSL [18], and FGGP [60] are considered to assess their advantages in representation learning. **Settings.** GeoFed performs the semi-supervised node classification task. In all experiments, we divide the global graph into 10 clients with the Louvain algorithm [4]. For each client’s local data, we adopt a category-balanced hierarchical partitioning strategy and use its default training/validation/test set ratio for different datasets to ensure that the proportion of each category in the training, validation, and test sets is consistent. We use a two-layer GCN as the base model, with a hidden dimension of 64 and a dropout rate of 0.5. The learning rate lr is 0.005. FGL training is performed for 200 rounds, and all clients participate in each round. Each client performs 4 local training epochs. The optimizer is Adam with a learning rate of 0.01 and weight decay of 5×10^{-4} . In GeoFed, the Ricci curvature and Poincaré embedding dimension are 1 and 2, and the temperature coefficient ranges from [0.5, 1]. Model evaluation is conducted using accuracy and F1 score as the metrics.

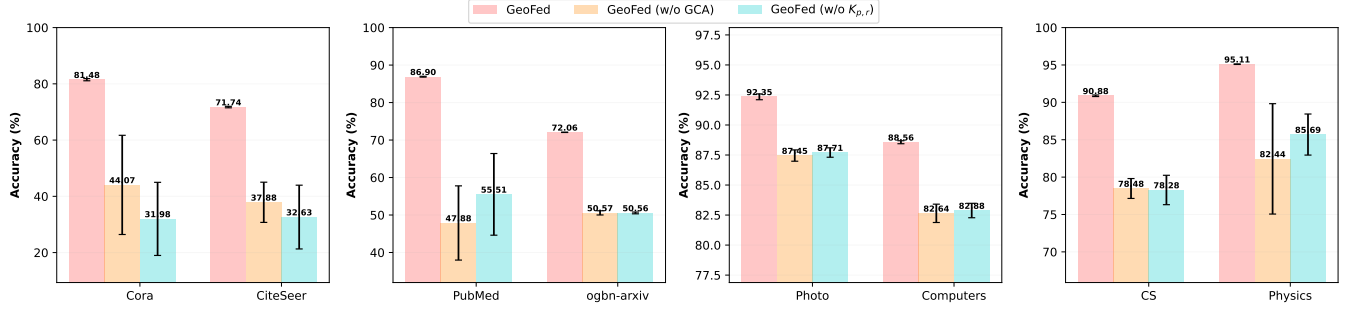


Figure 3: Ablation study. Server aggregation variants: average and geometric average.

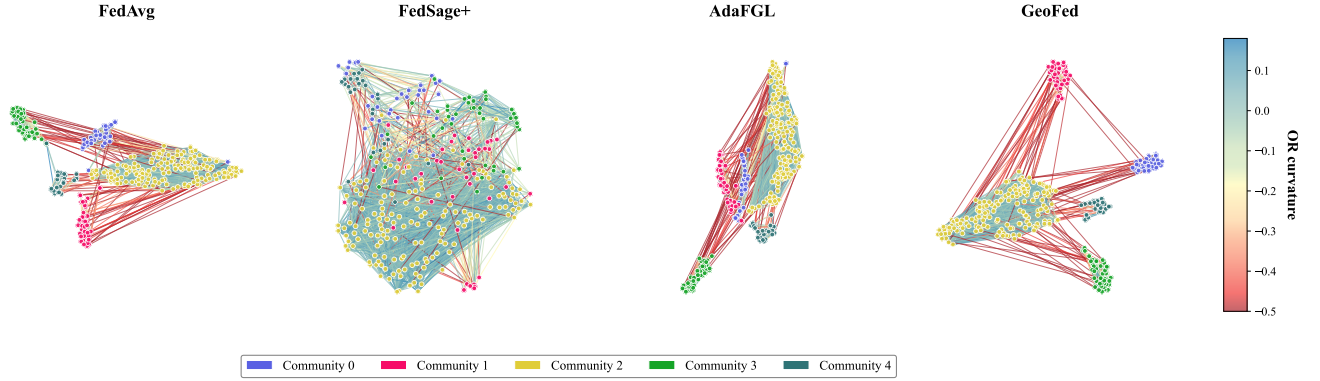


Figure 4: Case study on SBM dataset (a synthetic dataset generated from the Stochastic Block Model). GeoFed achieves the best results with both inter-class distinction and strong intra-class clustering.

5.2 RQ1: Performance Comparison

We compare the node classification performance of GeoFed against 10 state-of-the-art baselines to evaluate its effectiveness. Tab. 1 reports the classification Accuracy and F1 score, demonstrating that GeoFed significantly outperforms the baselines on most datasets, achieving greater stability under geometric heterogeneity. Notably, on the large-scale ogbn-arxiv graph (Gromov hyperbolicity [56] $\delta = 1.5$), where a lower hyperbolicity indicates a stronger hierarchical structure inherently conducive to geometric space modeling, GeoFed achieves a substantial improvement of over 7.4%. This advantage stems from the graph’s pronounced power-law distribution, which aligns well with our design to enable the robust selection of reliable source domains and effective topology-aware geometric aggregation. Conversely, on Cora ($\delta = 2.5$), the benefits are more constrained. Due to its strong assortativity, narrow curvature distribution, and relatively pure communities, standard Euclidean aggregation has approached its performance ceiling, limiting the additional gain derivable from geometric signals. However, the F1 score remains overall superior, indicating that GeoFed’s geometric weighting and personalization enhance overall discriminative quality, especially in scenarios with class imbalance or less distinct boundaries. Furthermore, our method achieves similar ACC on Photo as the strongest baseline, FedSage+, but achieves the best F1 score. This indicates that e-commerce graphs have

both non-stationary characteristics and a high number of cross-community edges. Our design can mitigate oversmoothing and cross-community friction in these scenarios.

5.3 RQ2: Ablation Study

We configured two variants of server aggregation to verify the importance of integrating geometric signals. *GeoFed (w/o GCA)* employs the most basic mean aggregation. *GeoFed (w/o $K_{p,r}$)* performs mean aggregation only on the uploaded geometric signals, which are then used for unified weighting of model parameters.

Fig. 3 presents the accuracy and standard deviation of the methods’ ablation results on 8 datasets. The results indicate: (1) The absence of geometry guidance significantly degrades performance, as mean aggregation forcibly aligns the local optima of geometric mismatches to the global level, leading to representational geometric mismatches and convergence oscillations. (2) Performing only geometric mean aggregation shows some improvement on datasets with high heterophily, but the effect is unstable. For CiteSeer, where the geometric signal distribution is relatively narrow, this method further flattens the distinction, causing edge or node weights to converge, which fails to aid local training and may even introduce noise. Overall, geometric averaging is not geometric consistency. We still need to filter personalization and weight fusion to leverage a few clients with strong geometric characteristics and achieve stable returns in geometrically heterogeneous distributed scenarios.

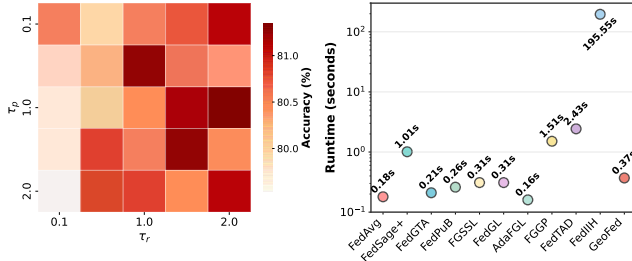


Figure 5: Hyperparameter and time analysis on Cora.

5.4 RQ3: Case Study

We use the SBM dataset as a case study. This dataset generates a graph with a predefined block structure and inter-block/intra-block connection probabilities, exhibiting clear community boundaries. Based on the structural characteristics of the SBM, we partition the graph using the METIS-Plus [31] algorithm (with 70 initial communities, a common technique in federated learning that does not affect the conclusions regarding model performance) and assign it to 3 clients. This results in the partitioned clients carrying blocks of the SBM, exhibiting strong geometric heterogeneity.

Fig. 4 and Fig. 1 visualize Client 1’s node representations after training and the original feature, which exhibits a strong geometric heterogeneity. GeoFed clearly outperforms other methods, effectively minimizing intra-class distances and sharpening inter-class boundaries via adaptive geometric weighting. In contrast, raw features lack distinct community boundaries; FedAvg achieves only rough separation, failing to cluster marginal nodes (blue) due to rigid averaging; FedSage+ suffers from generation noise under strong heterogeneity, diluting local distinctiveness; and AdaFGL ignores structural metrics, leading to knowledge conflicts between classes.

5.5 RQ4: Auxiliary Analysis

We conducted hyperparameter and time analysis on Cora.

Firstly, we analyzed the temperature coefficient **hyperparameters** $\tau_p, \tau_r \in [0.1, 2.0]$ during geometric weight fusion, as shown on the left side of Fig. 5. The model performs best around the moderate range. Low temperatures $\tau < 0.3$ easily lead to weight collapse, while high temperatures $\tau > 1.5$ tend towards uniformity, weakening geometric filtering. This indicates that values in the range of $[0.5, 1.0]$ are optimal. When the number of clients increases and the geometric quality differentiation is significant, τ can be slightly reduced to enhance selectivity.

For the average **time cost** of one round of communication, as shown in the right side of Fig. 5, GeoFed is comparable to FGSSL and FedGL methods, maintaining the same order of magnitude as most methods, while achieving excellent and stable performance with consideration of time cost. Specifically, GeoFed uses a lightweight MLP for client geometric heads, with inference costs approximately linear, and Ricci and Poincaré information is precomputed, introducing no additional manifold optimization costs. On the server side, geometric similarity and weight fusion involve tensor operations along the client dimension. The construction and weighted

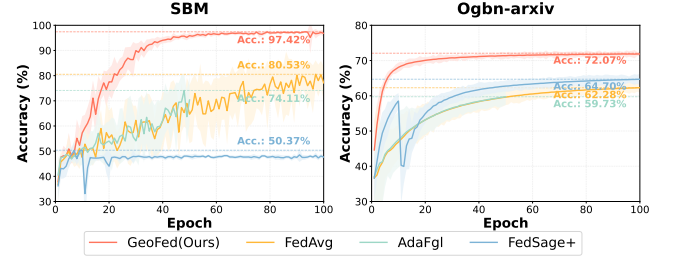


Figure 6: Convergence analysis.

dissemination of the personalization matrix is small in scale and overall can be considered negligible.

We conducted **convergence analysis** on the SBM and ogbn-arxiv datasets, which respectively offer distinctiveness for observation and large-scale graph expressiveness. Fig. 6 illustrates the convergence over 100 epochs for these datasets, with the shaded area representing the standard deviation across 10 runs. It can be observed that for datasets with significant client differences and strong long-range dependencies, GeoFed converges quickly, with accuracy rapidly climbing and reaching a stable phase earlier, achieving optimal convergence and accuracy. This indicates that its temperature fusion in the mid-to-high range results in sparser and more effective federated weights, reducing negative transfer from poor-quality clients to the global model, with noticeably less fluctuation in the curves. FedAvg converges more slowly, with accuracy rising gradually. FedSage+ struggles to handle strong heterogeneous information in the SBM dataset, resulting in delayed convergence. AdaFgl exhibits relatively noticeable accuracy fluctuations during convergence due to its two-stage training setup, where the second stage’s truncated training and re-aggregation lead to accuracy drops or even collapse. In summary, GeoFed achieves faster and more stable convergence through geometry-aware robust fusion and personalization, avoiding late-stage collapse in extremely heterogeneous scenarios.

6 Conclusion

This paper introduces GeoFed, a geometry-consistent federated graph learning framework, aiming to address the dual challenges of personalized geometric representation at the client side and global geometric alignment at the server side. The design of GeoFed comprises two components: a Geometric Infusion Local Learner, which transforms Ricci curvature and Poincaré embeddings into learnable edge and node weights within standard Euclidean message passing; and a Geometric Consistent Aggregator, which converts client geometric signals into stable fusion weights and similarity-driven personalization matrices. Extensive experiments on various benchmarks demonstrate consistent improvements in accuracy and robustness, achieving a consistent advantage over baseline methods. Future work could consider incorporating learnable curvature and end-to-end geometric estimators, as well as extensions to dynamic and hypergraphs.

Acknowledgments

The corresponding author is Chunming Hu. This work is supported in part by the National Key Research and Development Program of China (No. 2024YFB4505604) and the National Natural Science Foundation of China (No. 62462007, No. 62302023). We owe sincere thanks to all authors for their valuable efforts and contributions.

References

- [1] Xuming An, Li Shen, Han Hu, and Yong Luo. 2023. Federated learning with manifold regularization and normalized update reaggregation. *Advances in Neural Information Processing Systems* 36 (2023), 55097–55109.
- [2] Jinheon Baek, Wonyong Jeong, Jiongdao Jin, Jaehong Yoon, and Sung Ju Hwang. 2023. Personalized subgraph federated learning. In *International conference on machine learning*. PMLR, 1396–1415.
- [3] Frank Bauer, Jürgen Jost, and Shiping Liu. 2011. Ollivier-Ricci curvature and the spectrum of the normalized graph Laplace operator. arXiv:1105.3803
- [4] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.
- [5] Debora Calderola, Massimiliano Mancini, Fabio Galasso, Marco Ciccone, Emanuele Rodolà, and Barbara Caputo. 2021. Cluster-driven graph federated learning over multiple domains. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2749–2758.
- [6] Ines Chami, Zhitao Ying, Christopher Ré, and Jure Leskovec. 2019. Hyperbolic graph convolutional neural networks. *Advances in neural information processing systems* 32 (2019).
- [7] Chuan Chen, Ziyue Xu, Weibo Hu, Zibin Zheng, and Jie Zhang. 2024. FedGL: Federated graph learning framework with global self-supervision. *Information Sciences* 657 (2024), 119976.
- [8] Huiyuan Chen, Lan Wang, Yusan Lin, Chin-Chia Michael Yeh, Fei Wang, and Hao Yang. 2021. Structured graph convolutional networks with stochastic masks for recommender systems. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 614–623.
- [9] Frederik Diehl, Thomas Brunner, Michael Truong Le, and Alois Knoll. 2019. Towards graph pooling by edge contraction. In *ICML 2019 workshop on learning and reasoning with graph-structured data*.
- [10] Xingbo Fu, Zihan Chen, Binchi Zhang, Chen Chen, and Jundong Li. 2024. Federated graph learning with structure proxy alignment. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 827–838.
- [11] Xingcheng Fu, Yisen Gao, Yuecen Wei, Qingyun Sun, Hao Peng, Jianxin Li, and Xianxian Li. 2024. Hyperbolic geometric latent diffusion model for graph generation. *International Conference on Machine Learning* (2024).
- [12] Xingcheng Fu, Jianxin Li, Jia Wu, Qingyun Sun, Cheng Ji, Senzhang Wang, Jiajun Tan, Hao Peng, and Philip S Yu. 2021. ACE-HGNN: Adaptive curvature exploration hyperbolic graph neural network. In *2021 IEEE international conference on data mining (ICDM)*. IEEE, 111–120.
- [13] Xingbo Fu, Binchi Zhang, Yushun Dong, Chen Chen, and Jundong Li. 2022. Federated graph machine learning: A survey of concepts, techniques, and applications. *ACM SIGKDD Explorations Newsletter* 24, 2 (2022), 32–47.
- [14] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. 2020. An efficient framework for clustered federated learning. *Advances in neural information processing systems* 33 (2020), 19586–19597.
- [15] Albert Gu, Frederic Sala, Beliz Gunel, and Christopher Ré. 2018. Learning mixed-curvature representations in product spaces. In *International conference on learning representations*.
- [16] Caglar Gulcehre, Misha Denil, Mateusz Malinowski, Ali Razavi, Razvan Pascanu, Karl Moritz Hermann, Peter Battaglia, Victor Bapst, David Raposo, Adam Santoro, et al. 2018. Hyperbolic attention networks. *International conference on learning representations* (2018).
- [17] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [18] Wenke Huang, Guancheng Wan, Mang Ye, and Bo Du. 2024. Federated graph semantic and structural learning. In *Proceedings of the International Joint Conference on Artificial Intelligence*.
- [19] Yutao Huang, Lingyang Chu, Zirui Zhou, Lanjun Wang, Jiangchuan Liu, Jian Pei, and Yong Zhang. 2021. Personalized cross-silo federated learning on non-iid data. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 7865–7873.
- [20] TN Kipf. 2017. Semi-supervised classification with graph convolutional networks. *International conference on learning representations* (2017).
- [21] Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguná. 2010. Hyperbolic geometry of complex networks. *Physical Review E* 82, 3 (2010), 036106.
- [22] Marc Law, Renjie Liao, Jake Snell, and Richard Zemel. 2019. Lorentzian distance learning for hyperbolic representations. In *International Conference on Machine Learning*. PMLR, 3672–3681.
- [23] John M Lee. 2006. *Riemannian manifolds: an introduction to curvature*. Vol. 176. Springer Science & Business Media.
- [24] Mingxi Lei, Meng Ding, Chunwei Ma, Ziyun Huang, Mingchen Gao, and Jinhui Xu. 2025. Unifying domain gap in Federated Learning: A geometric approach. *Neurocomputing* 617 (2025), 128813.
- [25] Haifeng Li, Jun Cao, Jiawei Zhu, Yu Liu, Qing Zhu, and Guohua Wu. 2022. Curvature graph neural network. *Information Sciences* 592 (2022), 50–66.
- [26] Rui Li, Xin Yuan, Mohsen Radfar, Peter Marendy, Wei Ni, Terrence J O'Brien, and Pablo M Casillas-Espinosa. 2021. Graph signal processing, graph neural network and graph learning on biological data: a systematic review. *IEEE Reviews in Biomedical Engineering* (2021), 109–135.
- [27] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems* 2 (2020), 429–450.
- [28] Xiao Li, Li Sun, Mengjie Ling, and Yan Peng. 2023. A survey of graph neural network based recommendation in social networks. *Neurocomputing* (2023), 126441.
- [29] Xunkai Li, Zhengyu Wu, Wentao Zhang, Henan Sun, Rong-Hua Li, and Guoren Wang. 2024. Adafgl: A new paradigm for federated node classification with topology heterogeneity. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2517–2530.
- [30] Xunkai Li, Zhengyu Wu, Wentao Zhang, Yinlin Zhu, Rong-Hua Li, and Guoren Wang. 2023. Fedgta: Topology-aware averaging for federated graph learning. In *Proceedings of the VLDB Endowment*.
- [31] Xunkai Li, Yinlin Zhu, Boyang Pang, Guochen Yan, Yeyu Yan, Zening Li, Zhengyu Wu, Wentao Zhang, Rong-Hua Li, and Guoren Wang. 2024. Openfgl: A comprehensive benchmark for federated graph learning. arXiv:2408.16288
- [32] Tong Lin and Hongbin Zha. 2008. Riemannian manifold learning. *IEEE transactions on pattern analysis and machine intelligence* 30, 5 (2008), 796–809.
- [33] Yong Lin, Linyuan Lu, and Shing-Tung Yau. 2011. Ricci curvature of graphs. *Tohoku Mathematical Journal, Second Series* 63, 4 (2011), 605–627.
- [34] Jingxin Liu, Jieren Cheng, Renda Han, Wenxuan Tu, Jiaxin Wang, and Xin Peng. 2025. Federated graph-level clustering network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 18870–18878.
- [35] Qi Liu, Maximilian Nickel, and Douwe Kiela. 2019. Hyperbolic graph neural networks. *Advances in neural information processing systems* 32 (2019).
- [36] Rui Liu, Pengwei Xing, Zichao Deng, Anran Li, Cuntai Guan, and Han Yu. 2024. Federated graph neural networks: Overview, techniques, and challenges. *IEEE transactions on neural networks and learning systems* (2024).
- [37] Yijing Liu, Dongming Han, Jianwei Zhang, Haiyang Zhu, Mingliang Xu, and Wei Chen. 2022. Federated multi-task graph learning. *ACM Transactions on Intelligent Systems and Technology (TIST)* 13, 5 (2022), 1–27.
- [38] Ziyao Liu, Yu Jiang, Jiyuan Shen, Minyi Peng, Kwok-Yan Lam, Xingliang Yuan, and Xiaoning Liu. 2024. A survey on federated unlearning: Challenges, methods, and future directions. *Comput. Surveys* 57, 1 (2024), 1–38.
- [39] Jiayi Luo, Qingyun Sun, Yuecen Wei, Haonan Yuan, Xingcheng Fu, and Jianxin Li. 2026. Privacy auditing of multi-domain graph pre-trained model under membership inference attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [40] Yanbiao Ma, Wei Dai, Wenke Huang, and Jiayi Chen. 2025. Geometric knowledge-guided localized global distribution alignment for federated learning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 20958–20968.
- [41] Yao Ma, Xiaorui Liu, Neil Shah, and Jiliang Tang. 2021. Is homophily a necessity for graph neural networks? *International conference on learning representations* (2021).
- [42] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.
- [43] Chien-Chun Ni, Yu-Yao Lin, Jie Gao, Xianfeng David Gu, and Emil Saucan. 2015. Ricci curvature of the internet topology. In *2015 IEEE conference on computer communications (INFOCOM)*. IEEE, 2758–2766.
- [44] Chien-Chun Ni, Yu-Yao Lin, Feng Luo, and Jie Gao. 2019. Community detection on networks with Ricci flow. *Scientific reports* 9, 1 (2019), 9984.
- [45] Maximilian Nickel and Douwe Kiela. 2017. Poincaré Embeddings for Learning Hierarchical Representations. In *NeurIPS*. 6338–6347.
- [46] Maximilian Nickel and Douwe Kiela. 2018. Learning continuous hierarchies in the lorentz model of hyperbolic geometry. In *International conference on machine learning*. PMLR, 3779–3788.
- [47] Formerly Data Protection. 2018. General data protection regulation (GDPR). *Intersoft Consulting*, Accessed in October 24, 1 (2018).
- [48] Felix Sattler, Klaus-Robert Müller, and Wojciech Samek. 2020. Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints. *IEEE transactions on neural networks and learning systems* 32, 8 (2020), 3710–3722.

- [49] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *European semantic web conference*. Springer, 593–607.
- [50] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. arXiv:1811.05868
- [51] Lichao Sun and Lingjuan Lyu. 2020. Federated model distillation with noise-free differential privacy. In *Proceedings of the International Joint Conference on Artificial Intelligence* (2020).
- [52] Li Sun, Zhongbao Zhang, Junda Ye, Hao Peng, Jiawei Zhang, Sen Su, and Philip S Yu. 2022. A self-supervised mixed-curvature graph neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 4146–4155.
- [53] Rei Tamaru, Pei Li, and Bin Ran. 2025. Geo-ORBIT: A Federated Digital Twin Framework for Scene-Adaptive Lane Geometry Detection. arXiv:2507.08743
- [54] Yue Tan, Yixin Liu, Guodong Long, Jing Jiang, Qinghua Lu, and Chengqi Zhang. 2023. Federated learning on non-iid graphs via structural knowledge sharing. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 9953–9961.
- [55] Alexandru Tifrea, Gary Bécigneul, and Octavian-Eugen Ganea. 2019. Poincaré Glove: Hyperbolic Word Embeddings. In *ICLR*.
- [56] Alexandru Tifrea, Gary Bécigneul, and Octavian-Eugen Ganea. 2018. Poincaré glove: Hyperbolic word embeddings. arXiv preprint arXiv:1810.06546 (2018).
- [57] Abraham A Ungar. 2007. The hyperbolic square and mobius transformations. *Banach Journal of Mathematical Analysis* 1, 1 (2007), 101–116.
- [58] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. arXiv:1710.10903
- [59] Lilapati Waikhom and Ripon Patgiri. 2023. A survey of graph neural networks in various learning paradigms: methods, applications, and challenges. *Artificial Intelligence Review* (2023), 6295–6364.
- [60] Guancheng Wan, Wenke Huang, and Mang Ye. 2024. Federated graph learning under domain shift with generalizable prototypes. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 15429–15437.
- [61] Zixi Wang and M Cenk Gursoy. 2024. QMGeo: Differentially Private Federated Learning via Stochastic Quantization with Mixed Truncated Geometric Distribution. In *ICC 2024-IEEE International Conference on Communications*. IEEE, 2300–2305.
- [62] Yuecen Wei, Haonan Yuan, Xingcheng Fu, Qingyun Sun, Hao Peng, Xianxian Li, and Chunming Hu. 2024. Poincaré differential privacy for hierarchy-aware graph embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 9160–9168.
- [63] Lingfeng Wen, Xuan Tang, Mingjie Ouyang, Xiangxiang Shen, Jian Yang, Daxin Zhu, Mingsong Chen, and Xian Wei. 2024. Hyperbolic graph diffusion model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 15823–15831.
- [64] Di Wu, Jun Bai, Yiliao Song, Junjun Chen, Wei Zhou, Yong Xiang, and Atul Sajjanhar. 2024. FedInverse: Evaluating privacy leakage in federated learning. In *The twelfth international conference on learning representations*.
- [65] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* (2020), 4–24.
- [66] Han Xie, Jing Ma, Li Xiong, and Carl Yang. 2021. Federated graph classification over non-iid graphs. *Advances in neural information processing systems* 34 (2021), 18839–18852.
- [67] Bingbing Xu, Huawei Shen, Bing-Jie Sun, Rong An, Qi Cao, and Xueqi Cheng. 2021. Towards Consumer Loan Fraud Detection: Graph Neural Networks with Role-Constrained Conditional Random Field. In *AAAI*. 4537–4545.
- [68] Yangkai Xue, Jindou Dai, Zhipeng Lu, Yuwei Wu, and Yunde Jia. 2024. Residual hyperbolic graph convolution networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 16247–16254.
- [69] Yihan Yan, Shen Wang, Fanghui Sun, and Xiaojun Tong. 2024. Personalized federated learning with multi-view geometry structure. *IEEE Internet of Things Journal* (2024).
- [70] Menglin Yang, Harshit Verma, Delvin Ce Zhang, Jiahong Liu, Irwin King, and Rex Ying. 2024. Hypformer: Exploring efficient transformer fully in hyperbolic space. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3770–3781.
- [71] Menglin Yang, Min Zhou, Zhihao Li, Jiahong Liu, Lujia Pan, Hui Xiong, and Irwin King. 2023. Hyperbolic graph neural networks: A review of methods and applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 5843–5844.
- [72] Menglin Yang, Min Zhou, Lujia Pan, and Irwin King. 2023. khgn: Tree-likeness modeling via continuous and discrete curvature learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2965–2977.
- [73] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.
- [74] Wentao Yu, Shuo Chen, Yongxin Tong, Tianlong Gu, and Chen Gong. 2025. Modeling inter-intra heterogeneity for graph federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 22236–22244.
- [75] Chengkun Zhang and Junbin Gao. 2021. Hype-han: Hyperbolic hierarchical attention network for semantic embedding. In *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*. 3990–3996.
- [76] Guixian Zhang, Guan Yuan, Debo Cheng, Lin Liu, Jiuyong Li, and Shichao Zhang. 2025. Mitigating propensity bias of large language models for recommender systems. *ACM Transactions on Information Systems* 43, 6 (2025), 1–26.
- [77] Ke Zhang, Carl Yang, Xiaoxiao Li, Lichao Sun, and Siu Ming Yiu. 2021. Sub-graph federated learning with missing neighbor generation. *Advances in neural information processing systems* 34 (2021), 6671–6682.
- [78] Yiding Zhang, Xiao Wang, Chuan Shi, Xunqiang Jiang, and Yanfang Ye. 2021. Hyperbolic graph attention network. *IEEE Transactions on Big Data* 8, 6 (2021), 1690–1701.
- [79] Yupei Zhang, Shuangshuang Wei, Shuhui Liu, Yifei Wang, Yunan Xu, Yuxin Li, and Xuequn Shang. 2022. Graph-regularized federated learning with shareable side information. *Knowledge-Based Systems* 257 (2022), 109960.
- [80] Longfei Zheng, Jun Zhou, Chaochao Chen, Bingzhe Wu, Li Wang, and Benyu Zhang. 2021. ASFGNN: Automated separated-federated graph neural network. *Peer-to-Peer Networking and Applications* 14, 3 (2021), 1692–1704.
- [81] Pengyang Zhou, Chaochao Chen, Weiming Liu, Xinting Liao, Fengyuan Yu, Zhihui Fu, Xingyu Lou, Wu Wen, Xiaolin Zheng, and Jun Wang. 2025. FedGF: Enhancing Structural Knowledge via Graph Factorization for Federated Graph Learning. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*. 448–456.
- [82] Mike He Zhu, Léna Néhale Ezzine, Dianbo Liu, and Yoshua Bengio. 2022. FedILC: Weighted geometric mean and invariant gradient covariance for federated learning on non-IID data. arXiv:2205.09305
- [83] Yinlin Zhu, Xunkai Li, Zhengyu Wu, Di Wu, Miao Hu, and Rong-Hua Li. 2024. Fedtd: Topology-aware data-free knowledge distillation for subgraph federated learning. In *Proceedings of the International Joint Conference on Artificial Intelligence*.

A Algorithm

The overall process of GeoFed is shown in Algorithm 1.

Algorithm 1: The overall training process of GeoFed

Input: initial parameter θ^0 ; curvature c ; local steps E ; communication rounds T ; set of selected clients $\mathcal{S}_k$; temperature coefficient τ_p, τ_r ; local and global learning rate η_l, η_g .

Output: global personalized parameters θ_t .

```

1 Server init:  $\Delta^0 \leftarrow 0$ , broadcast  $\theta^0$ ;
2 for  $t = 0, 1, \dots, T - 1$  do
3   Server broadcasts  $\theta^t$  and  $\Delta^t$  to clients in  $\mathcal{S}_t$ ;
4   for each client  $i \in \mathcal{S}_t$  in parallel do
5     Local init:  $\theta_i^{t,0} \leftarrow \theta^t$ ;
6     // Compute geometric signals on local graph
7     Compute OR curvature  $\kappa(u, v)$  and Poincaré embedding  $\mathbf{z}_v \leftarrow \text{Eq. (4), (6)}$ ;
8     for  $e = 0, 1, \dots, E - 1$  do
9       Compute distance weights  $\tilde{\omega}_{uv}^e$  and hierarchical weights  $\omega_v^p \leftarrow \text{Eq. (5), (7)}$ ;
10      Get node representation  $\mathbf{h}_v \leftarrow \text{Eq. (8)}$  with geometric-aware message passing;
11      Compute loss  $\mathcal{L} \leftarrow \text{Eq. (9)}$  and update  $\theta_i^{t,e+1} \leftarrow \theta_i^{t,e} - \eta_l \nabla f_i(\theta_i^{t,e})$ ;
12    end
13    Compute  $\Delta_i^t \leftarrow \theta_i^{t,E} - \theta^t$ ,  $\omega_{uv}^r \leftarrow \phi_r(\mathbf{z}_{u,v})$ , and  $\omega_v^p \leftarrow \phi_p(\mathbf{z}_v)$ ;
14  end
15  Upload  $(\Delta_i^t, \omega_{uv}^r, \omega_v^p)$  to server;
16  // Geometric neighborhood personalization
17  Compute the personalization matrix  $K_p[i, j]$  and  $K_r[i, j] \leftarrow \text{Eq. (10)}$  between clients;
18  Aggregate  $\Delta^{t+1} \leftarrow \sum_{i \in \mathcal{S}_t} \sigma(K_p \odot K_r) \Delta_i^t$ ;
19  Update global parameter  $\theta_j^{t+1} \leftarrow \theta^{t+1} - \eta_g \sum_{i \in \mathcal{S}_t} K_{p,r}[i, j] \Delta_i^t$ .
20 end

```

B Experimental details

In this section, we introduce the statistical characteristics of the benchmark datasets, provide a detailed description of the state-of-the-art baselines used for comparison, and specify the hardware and software configurations.

B.1 Dataset Statistics

To comprehensively evaluate performance across diverse graph modalities, we conduct experiments on eight standard benchmark datasets spanning three distinct domains: citation networks, co-purchase networks, and co-authorship networks. Detailed statistics for each dataset are summarized in Table 2.

- **Citation Networks:** We utilize three standard benchmark datasets: Cora, CiteSeer, and PubMed [73]. In these graphs,

nodes correspond to scientific publications, and edges encode citation links. The node features are sparse bag-of-words vectors representing the presence of specific terms. To assess performance on large-scale graph learning, we also include ogbn-arxiv [17], a directed graph derived from the Microsoft Academic Graph (MAG). Unlike the standard baselines, node features in ogbn-arxiv are 128-dimensional dense embeddings, obtained by averaging Word2Vec representations of paper titles and abstracts to capture rich semantic context.

- **Co-purchase Networks:** For e-commerce scenarios, we employ Amazon Photo and Amazon Computers [50]. These are subsets of the Amazon co-purchase graph, where nodes represent items and edges signify that two products are frequently bought together. Node attributes are constructed from product reviews using bag-of-words representations.
- **Co-authorship Networks:** We incorporate Coauthor CS and Coauthor Physics [50] to model social collaboration. Derived from the MAG, these graphs define nodes as authors and edges as co-authorship relationships. Node features are generated from the keywords of the authors' papers, and the task involves classifying authors into their respective active research fields.

Table 2: Statistics of datasets.

Dataset	#Nodes	#Edges	#Features	#Classes
Cora	2,708	5,429	1,433	7
CiteSeer	3,327	4,732	3,703	6
PubMed	19,717	44,338	3,703	3
ogbn-arxiv	169,343	1,166,243	128	40
Photo	7,650	238,162	745	8
Computers	13,752	491,722	767	10
CS	18,333	163,788	6,805	15
Physics	34,493	495,924	8,415	5

B.2 Baseline details

We compare our proposed framework against a representative set of state-of-the-art baselines, including the typical federated learning algorithm and advanced federated graph learning methods.

- **FedAvg** [42] enables decentralized model training across diverse devices while preserving data privacy. In this framework, a central server distributes a global model to clients, where they independently execute local updates using stochastic gradient descent. These updates are then aggregated by the server via averaging to refine the global model, with the cycle repeating until convergence.
- **FedSage+** [77] integrates node features, link structures, and labels by combining a GraphSage model with FedAvg across local subgraphs. It further extends this approach by training a generator to recover missing links, thereby enhancing the robustness and completeness of the model.
- **FedGTA** [30] merges large-scale graph learning with federated settings by encoding topology and node attributes at the client level. Clients compute local smoothing confidence and mixed moments of neighbor features, which are

uploaded to the server. The server then utilizes these metrics as weights for personalized model aggregation to ensure effective integration.

- **Fed-PUB [2]** computes similarities between local GNNs using functional embeddings derived from random graph inputs to facilitate weighted server-side aggregation. Additionally, it employs a personalized sparse mask to selectively update parameters relevant to each client's subgraph.
- **FGSSL [18]** addresses local client distortion by focusing on node-level semantics and graph-level structures via contrastive loss functions. It enhances discrimination by aligning local nodes with global counterparts of the same class. Furthermore, it transforms adjacency relationships into similarity distributions, using the global model to distill relational knowledge into local models.
- **FedGL [7]** utilizes global self-supervision information to enhance prediction accuracy. Based on the prediction outcomes and node embeddings uploaded by clients, the server derives global pseudo labels and a global pseudo graph. These global insights are distributed to clients to augment training labels and refine graph structures, improving local model performance.
- **AdaFGL [29]** proposes a two-step personalized approach where multi-client models are first aggregated into a federated knowledge extractor at the server. Subsequently, each client performs personalized training utilizing both the local subgraph and the guidance from this federated extractor.
- **FGGP [60]** divides the global model into a classifier and a feature extractor linked by prototypes. It replaces traditional classifiers with clustered prototypes to enhance class discrimination and utilizes contrastive learning to imbue these prototypes with global knowledge, thereby improving model generalization.
- **FedTAD [83]** computes topology-aware node embeddings to evaluate the reliability of class-wise knowledge. Guided by this reliability, the server conducts data-free knowledge distillation to selectively transfer reliable knowledge from local models across multiple clients to the global model.
- **FedIIIH [74]** proposes a hierarchical variational graph autoencoder to infer the complete distribution of subgraph data from a global perspective, enabling accurate inter-subgraph similarity calculation via divergences. Furthermore, it disentangles local subgraphs into multiple latent factors to capture intra-heterogeneity, facilitating factor-wise separate federation for personalized model aggregation.

B.3 Hardware and Software Information

- **Operating system:** Ubuntu 22.04.5 LTS operating system.
- **Hardware:** Dual Intel(R) Xeon(R) Gold 6148 CPUs @ 2.40GHz and 32Gi memory, and NVIDIA A100-SXM4-40GB GPU.
- **Software:** CUDA 12.1, Python 3.10.18, PyTorch 2.1.2+cu121, PyTorch-Geometrics 2.6.1.